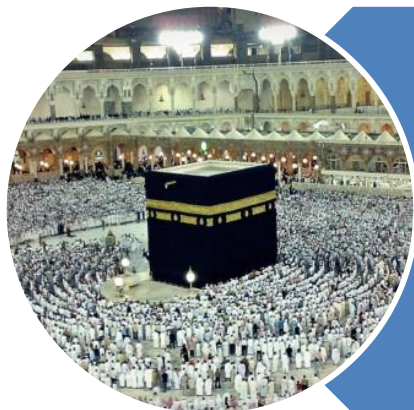


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Table

Introduction

C

P

C

S

2

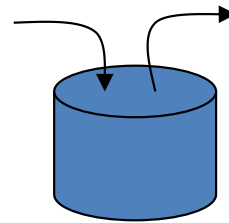
0

4

3

Table - Introduction

- Arrays
- Linked List
- Stacks
- Queues
- Tree
- Graph
- **Table**



0	:	:
150	842335821	Andy
:	:	:
368	527447521	Betty
:	:	:
527	452856829	David
:	:	:
:	:	:
884	651543218	Bill
:	:	:
999	:	:

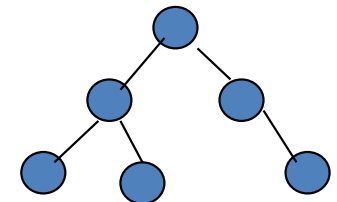
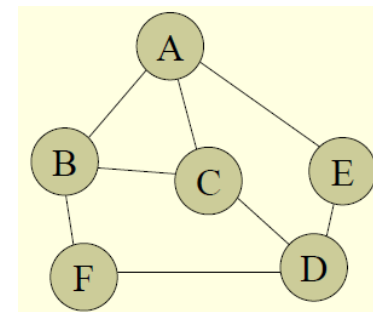


Table - Introduction

- Definition
 - An **abstract data type** that stores & retrieves **records** according to their search key values
- Record
 - Each individual **row** in the table
- Example
 - **Attendance** Sheet of class
 - Each record shown on which day the student was present or absent
 - A **database** of student records
 - So each record will have a pid, first name, last name, SSN, address, phone, email, etc.



sid (key)	name	score
0012345	andy	81.5
0033333	betty	90
0056789	david	56.8

...

9903030	tom	73
9908080	bill	49

...

C

P

C

S

2

0

4

Table - Introduction

- Basic Operations
 - **Adding** Record
 - **Deleting** Record
 - **Searching** Record

Table Implementation

C

P

C

S

2

0

4

7

Table - Implementation

- Arrays – Unsorted
 - Adding Record
 - **$O(1)$** since we simply add at the end of the unsorted array
 - Deleting Record
 - Very slow, or **$O(n)$** , since we have to search through the entire array to find the desired record to delete
 - Searching Record
 - Very slow, or **$O(n)$** , since we search through the entire table

C

P

C

S

2

0

4

Table - Implementation

- Arrays – Sorted
 - **Adding** Record
 - Must insert at correct position
 - And then ALL other records, after insertion spot, must be moved
 - Very slow, or $O(n)$
 - **Deleting** Record
 - Must find the record to delete, $O(n)$
 - Must fill the “hole”, which means moving all other items, $O(n)$
 - **Searching** Record
 - Binary search!
 - Fast, or $O(\log n)$

C

P

C

S

2

0

4

Table - Implementation

- Binary Search Tree
 - **Adding** Record
 - Inserting into proper position in BST
 - Fast, or **$O(\log n)$**
 - **Deleting** Record
 - Must find correct position to delete
 - Fast, or **$O(\log n)$**
 - **Searching** Record
 - Also Fast, or **$O(\log n)$**

C

P

C

S

2

0

4

Table - Implementation

- The best possible so far is **Binary Search Tree**
- There is an other way out
 - O (1)
- Direct Access Table**
 - 7, 36, 0, 19, 25, 14, 42, 75

0	...	7		14	...	19	...	25	...	36	...	42	...	75	...	100
0		7		14		19		25		36		42		75		

C

P

C

S

2

0

4

Table - Implementation

- Direct Access Table
 - Adding Record
 - $O(1)$
 - Deleting Record
 - $O(1)$
 - Searching Record
 - $O(1)$

0	...	7		14	...	19	...	25	...	36	...	42	...	75	...	100
0		7		14		19		25		36		42		75		

C

P

C

S

2

0

4

Table - Implementation

• Direct Access Table - Problem

- Let's say we want to store attendance record of **30 students** based on their **university ID**
- University ID is 7 digits long at KAU i.e. **1847478**
- We need an array of size; **10 million**
- So it will not be **O(1)**
- It would be **O(stupid)**

0	...	7		14	...	19	...	25	...	36	...	42	...	75	...	100
0		7		14		19		25		36		42		75		

Table - Implementation

- Direct Access Table - Solution
 - If we want to store attendance record of **30 students** based on their **university ID**
 - We should need a **reasonable size** array
 - Say it could be of **size 30 – 50**
 - **30 is ideal** but for future use **50 is also reasonable** as compared to 10 millions
- **Hash Table**

Hash Table

Hash Table - Introduction

- An **array** of table items, where the index is calculated by a **hash function**
 - Insertion and searching do **not** follow **standard protocol**
 - **Hash function** is used to **map** the key to the location in the table
 - A way to access a table (array) in relatively constant (quick) time is known as **Hashing**
 - In ideal situation all operations completed in **$O(1)$**

C

P

C

S

2

0

4

Hash Table – Hash Function

- A mathematical calculation that maps the search key to an index in a hash table
- Example:
 - Marking attendance in class
 - Rather than calling complete ID i.e. 1847478
 - We may use the last 3 digit i.e. 478 for each ID
 - Here extracting last three digit is a Hash Function

C

P

C

S

2

0

4

Hash Table – Hash Function

- A hash function is written $h(x)=i$
 - h is the name of the hash function
 - x is the record search key
 - Such as the student ID in our example
 - i is the output of the hash function
 - which refers to an index in the array (hash table)
 - Let's say we are trying to add to a hash table
 - Once i is calculated, we can then add the record at **HashTable [i]**

C

P

C

S

2

0

4

Hash Functions— **Introduction**

- There is **no standard way** of defining Hash Function
- **User** can define it as per his/her need
- Simple **Hash Functions** for integer values
 - Selecting Digits
 - Folding Method
 - Modulo Arithmetic

Hash Functions

Integer Values

C

P

C

S

2

0

4

Hash Functions— **Selecting Digits**

- Instead of using the whole integer, only **select several digits**
 - For example, if you have the ID # 123456789
 - just use the **first 3 digits**
 - $h(123456789)=123$
 - This is like the example we already did
- Fast & easy to calculate, but usually does not distribute randomly

C

P

C

S

2

0

4

Hash Functions— **Folding Method**

- Add the digits of the integer together
 - For example, if you have the ID #123456789
 - add all the digits together
 - $h(123456789) = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$ with hash table index range $0 \leq h(\text{search key}) \leq 81$
- Can add in different ways for hash tables of different sizes
 - $h(123456789) = 123 + 456 + 789 = 1368$ with hash table index range $0 \leq h(\text{search key}) \leq 2997$

C

P

C

S

2

0

4

Hash Functions— **Modulo Arithmetic**

- Using modulus as a hash function
 - $h(x) = x \bmod \text{tableSize}$
 - For $\text{tableSize} = 31$,
 $h(123456789) = 123456789 \bmod 31 = 2$
with hash table index range $0 \leq h(\text{search key}) \leq 30$
- Using a prime number as tableSize reduces chances of duplication

C

P

C

S

2

0

4

Hash Functions— **Non-Integers Keys**

- Ways to convert a string to an integer
 1. Assign **A to Z** the numbers **0 to 25**, and add the integers together
 - **String Key** is “ABC”
 - **Add integer** values $0 + 1 + 2$
 - **Integer key** will be 3

C

P

C

S

2

0

4

Hash Functions— **Non-Integers Keys**

- Ways to convert a string to an integer
 2. Use the **ASCII** or **Unicode** integer value for each character, and add the integers together
 - **String Key** is “ABC”
 - **Use ASCII code** values $65 + 66 + 67$
 - **Integer key** will be 198

C

P

C

S

2

0

4

Hash Functions— **Non-Integers Keys**

- Ways to convert a string to an integer
 3. Use the **binary number** for the **ASCII** or **Unicode** integer value for each character, and **concatenate** the **binary numbers** together
 - **String Key** is “ABC”
 - **Use binary numbers** 01000001 + 01000010 + 01000011 = 010000010100001001000011
 - **Integer key** will be 4276803

C

P

C

S

2

0

4

Hash Functions— Problem

- **Collision**

- When a hash function maps two or more search keys into the same location in the hash table

- $h(\text{key1}) = h(\text{key2})$, so have the same index value

- Example

- Hash Function $h(x) = x \bmod 31$

- $h(12345678) = h(26508090) = 21$

- This is collision

C

P

C

S

2

0

4

Collision – Resolution

- Two main approaches to collision resolution
 - Open Addressing
 - Linear Probing
 - Quadratic Probing
 - Double Hashing
 - Restructure the Hash Table
 - Bucket Hashing
 - Separate Chaining

Collision Resolution

Open Addressing

(Probing)

C

P

C

S

2

0

4

Open Addressing – **Linear Probing**

- In the case of a collision, keep going to the next hash table location until find an open location
 - In other words, if $\text{table}[i]$ is occupied, check $\text{table}[i+1]$, $\text{table}[i+2]$, $\text{table}[i+3]$, ...
 - Need 3 states for each hash table location: empty, occupied, deleted
- Common problem
 - Items tend to cluster together in the hash table

C

P

C

S

2

0

4

Open Addressing – Linear Probing

- Table size = 31
- Hash function = key mod 31
 - $h(1234) = 25$ $table[25] = 1234$
 - $h(4055) = 25+1$ $table[26] = 4055$
 - $h(3962) = 25+2$ $table[27] = 3962$
 - $h(5853) = 25+3$ $table[28] = 5853$
 - $h(1766) = 30$ $table[30] = 1766$
 - $h(1270) = 30+1$ $table[0] = 1270$ (wraps around)
 - All other table entries are empty

C

P

C

S

2

0

4

Open Addressing – Quadratic Probing

- Instead of checking the next location sequentially, check the next location based on a sequence of squares
 - In other words, if $\text{table}[i]$ is occupied, check $\text{table}[i+1^2]$, $\text{table}[i+2^2]$, $\text{table}[i+3^2]$, ...
 - Still have clustering (called “secondary clustering”), but this method is not as problematic as linear probing

Open Addressing – Quadratic Probing

- Table size = 31
- Hash function = $\text{key} \bmod 31$
 - $h(1234) = 25$ $\text{table}[25] = 1234$
 - $h(4055) = 25 + 1^2$ $\text{table}[26] = 4055$
 - $h(3962) = 25 + 2^2$ $\text{table}[29] = 3962$
 - $h(5853) = 25 + 3^2$ $\text{table}[3] = 5853$ (wraps around)
 - $h(1766) = 30$ $\text{table}[30] = 1766$
 - $h(1270) = 30 + 1^2$ $\text{table}[0] = 1270$ (wraps around)
 - All other table entries are empty

C

P

C

S

2

0

4

Open Addressing – **Double Hashing**

- Use two hash functions
 - First to find hash address
 - second hash function determines the step size in case of collision
- Some restrictions
 - $h_2(\text{searchKey}) \neq 0$ (step size should not be zero)
 - $h_2 \neq h_1$ (avoids clustering)

Open Addressing – **Double Hashing**

- Table size = 31
- Hash function #1 = $\text{key} \bmod 31$
- Hash function #2 = $23 - (\text{key} \bmod 23)$
 - $h1(1234) = 25$ $\text{table}[25] = 1234$
 - $h1(4055) = 25, h2(4055) = 16 (+25), \text{table}[10] = 4055$
 - $h1(3962) = 25, h2(3962) = 17 (+25), \text{table}[11] = 3962$
 - $h1(5853) = 25, h2(5853) = 12 (+25), \text{table}[6] = 5853$
 - $h1(1766) = 30$ $\text{table}[30] = 1766$
 - $h1(1270) = 30, h2(1270) = 18 (+30), \text{table}[17] = 1270$
 - All other table entries are empty

C

P

C

S

2

0

4

Open Addressing – **Double Hashing**

- $h1(key) = key \bmod 13$
- $h2(key) = 11 - (key \bmod 11)$
 - If key = 30, probe sequence would be 4, 7, 10, 0, 3, 6, 9, 12, 2, 5, 8, 11, 1 (step 3 each time)
 - If key = 50, probe sequence would be 11, 3, 8, 0, 5, 10, 2, 7, 12, 4, 9, 1, 6 (step 5 each time)

C

P

C

S

2

0

4

Open Addressing – Summary

- If table size is prime, then probe sequence will visit all table locations
- With open addressing, increasing table size will reduce collisions
 - When increasing the size, the hash function needs to be reapplied to every item in the old hash table to place it in the new hash table

Collision Resolution

Restructuring

(Hash Table)

C

P

C

S

2

0

4

Restructuring – Introduction

- The structure of the hash table is changed so that the same index location can store multiple items
- Two ways to restructure a hash table for collision resolution
 1. Bucket hashing
 2. Separate chaining

C

P

C

S

2

0

4

Restructuring – **Bucket Hashing**

- A hash table that has an array at each location $table[i]$, so that items of the same hash index are stored here
- Choosing the size of the bucket is problematic
 - If too small, will have collisions
 - If too big, will waste space

Restructuring – **Bucket Hashing**

- Table size = 31
- Hash function = $\text{key} \bmod 31$
 - $h(1234) = 25$ $\text{table}[25][0] = 1234$
 - $h(4055) = 25$ $\text{table}[25][1] = 4055$
 - $h(3962) = 25$ $\text{table}[25][2] = 3962$
 - $h(5853) = 25$ $\text{table}[25][3] = 5853$
 - $h(1766) = 30$ $\text{table}[30][0] = 1766$
 - $h(1270) = 30$ $\text{table}[30][1] = 1270$
 - All other table entries are empty

C

P

C

S

2

0

4

Restructuring – **Separate Chaining**

- A hash table that has linked list (a chain) at each location $\text{table}[i]$, so that items of the same hash index are stored here
- Size of the table is dynamic
 - Less problematic than static bucket implementation

Restructuring – **Separate Chaining**

- Table size = 31
- Hash function = $\text{key} \bmod 31$
 - $h(1234) = 25$, $\text{table}[25] \Rightarrow 1234$
 - $h(4055) = 25$, $\text{table}[25] \Rightarrow 4055 \Rightarrow 1234$
 - $h(3962) = 25$, $\text{table}[25] \Rightarrow 3962 \Rightarrow 4055 \Rightarrow 1234$
 - $h(5853) = 25$,
 $\text{table}[25] \Rightarrow 5853 \Rightarrow 3962 \Rightarrow 4055 \Rightarrow 1234$
 - $h(1766) = 30$, $\text{table}[30] \Rightarrow 1766$
 - $h(1270) = 30$, $\text{table}[30] \Rightarrow 1270 \Rightarrow 1766$

Hashing Example

Review Question

C

P

C

S

2

0

4

Question

- Insert the following numbers (in the order they are shown i.e. from left to right) into a hash table with an array of size 10, using the hash function;

- $H1(x) = x \bmod 10.$

1837, 2209, 7777, 2100, 6897, 4012

- Show the result of the insertions when hash collisions are resolved through:
 - a) Quadratic Probing
 - b) Double hashing
 - The second hash function used is $H2(x) = 4 - (x \bmod 2)$
 - c) Separate chaining.
 - Insert at the **END** of the linked list.

Quadratic Probing (1837, 2209, 7777, 2100, 6897, 4012)

Index	
0	2100
1	6897
2	4012
3	
4	
5	
6	
7	1837
8	7777
9	2209

$$\begin{aligned}
 H1(x) &= x \bmod 10 \\
 &= 1837 \bmod 10 \\
 &= 7
 \end{aligned}$$

Quadratic Probing

$$h + 1 = 8$$

$$h + 2^2 = h + 4$$

Double Hashing (1837, 2209, 7777, 2100, 6897, 4012)

Index	
0	7777
1	
2	4012
3	6897
4	2100
5	
6	
7	1837
8	
9	2209

$$H1(x) = x \bmod 10$$

$$= 1837 \bmod 10$$

$$= 7$$

Double Hashing

$$H2(x) = 4 - (x \bmod 2)$$

$$= 4 - (7777 \bmod 2)$$

$$= 4 - 1$$

$$= 3$$

Double Hashing

$$H2(x) = 4 - (x \bmod 2)$$

$$H2(x) = 4 - (2100 \bmod 2)$$

$$H2(x) = 4 - 0$$

$$H2(x) = 4$$

Double Hashing

$$H2(x) = 4 - (x \bmod 2)$$

$$= 4 - (6897 \bmod 2)$$

$$= 4 - 1$$

$$= 3$$

Separate Chaining (1837, 2209, 7777, 2100, 6897, 4012)

Index	
0	2100
1	
2	4012
3	
4	
5	
6	
7	1837 → 7777 → 6897
8	
9	2209

$$\begin{aligned}
 H1(x) &= x \bmod 10 \\
 &= 1837 \bmod 10 \\
 &= 7
 \end{aligned}$$

C

P

C

S

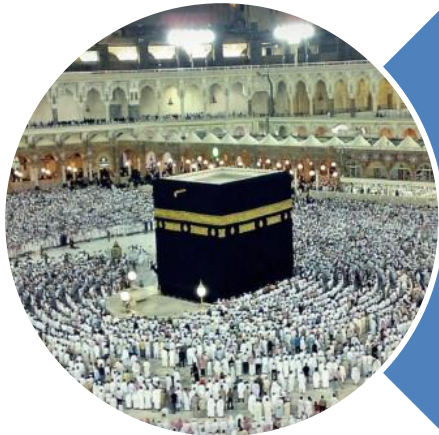
2

0

4

Hash Table – Summary

- We use a hash table to accomplish $O(1)$ access time into a table
 - While keeping the table to a reasonable size
 - Use a hash function to map the record “keys” into an index in the hash table
 - Collisions are bound to happen and are taken care of using several possible methods
- Comparison of Implementations (slowest to quickest)
 - Linear probing, quadratic probing, double hashing, separate chaining



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**